

Branch and bound for integer optimization

Michel Bierlaire

Algorithm and examples

```
[1]: import numpy as np
```

The branch and bound algorithm for integer optimization relies on the simplex algorithm to solve the relaxation of each subproblem. For the notebook to be self-contained, we include the code of the algorithm below, so that it can be called by the branch and bound algorithm.

1 Simplex algorithm

```
[2]: def simplexTableau(tableau):  
    """  
    :param tableau: the first simplex tableau  
    :type tableau: pandas dataframe  
  
    :return: p, q, opt, bounded where  
        - p is the column of the variable that must enter the basis, or  $\perp$   
     $\rightarrow$  None,  
        - q is the row of the variable that must leave the basis, or None,  
        - opt is True if the tableau is optimal (in this case, p and q  $\perp$   
     $\rightarrow$  are None)  
        - bounded is True if basic direction is unbounded (in this case,  $\perp$   
     $\rightarrow$  p and q are None)  
    :rtype: int, int, bool, bool  
    """  
    mtab, ntab = tableau.shape  
    m = mtab - 1  
    n = ntab - 1  
  
    reducedCost = tableau[-1, : -1]  
    # Identify the negative reduced costs  
  
    negativeReducedCost = reducedCost < -np.finfo(float).eps  
    if not negativeReducedCost.any():  
        # The tableau is optimal  
        return None, None, True, True
```

```

    # In Python, True is larger than False. The next statement returns the
    # index of a True entry in the array, that is the index of a negative
    →reduced cost.
    # It is the index of the variable that will enter the basis.
    p = np.argmax(negativeReducedCost)

    # Calculate the maximum step that can be done along the basic direction d[p]
    xb = tableau[:-1, -1]
    minusd = tableau[:-1, p]
    steps = np.array([xb[k] / minusd[k] if minusd[k] > 0 else np.inf for k in
    →range(m)])
    q = np.argmin(steps)
    step = steps[q]
    if step == np.inf:
        # The tableau is unbounded
        return None, None, False, False
    else:
        return p, q, False, True

```

```

[3]: def pivoting(tableau, p, q):
    """
    :param tableau: valid simplex tableau
    :type tableau: numpy.array 2D

    :param p: columns of the pivot
    :type p: int

    :param q: row of the pivot
    :type q: int

    :return: pivoted tableau
    :rtype: numpy.array 2D
    """
    m, n = tableau.shape
    if q >= m:
        raise Exception(f'The row of the pivot ({q}) must be between 0 and
    →{m-1}')
    if p >= n:
        raise Exception(f'The column of the pivot ({p}) must be between 0 and {n
    →- 1}')
    thepivot = tableau[q][p]
    if np.abs(thepivot) < np.finfo(float).eps:
        print(f'Tableau: {tableau}')
        print(f'Row {q} Col {p}')
        raise Exception(f'The pivot is too close to zero: {thepivot}')
    thepivotrow = tableau[q, :]
    newtableau = np.empty(tableau.shape)

```

```

newtableau[q, :] = tableau[q, :] / thepivot
for i in set(range(m)) - {q}:
    newtableau[i, :] = tableau[i, :] - tableau[i][p] * thepivotrow / thepivot
return newtableau

```

```

[4]: def prettyTableau(tableau):
    m, n = tableau.shape
    s = ''
    for i in range(m - 1):
        formattedRow = ['{:6.2g}'.format(k) for k in tableau[i, :-1]]
        s += '\t'.join(formattedRow).format(*tableau[i, :-1])
        s += f'|{tableau[i, -1]:6.2f}'
        s += '\n'
    for i in range(n):
        s += '-----\t'
    s += '\n'
    formattedRow = ['{:6.2g}'.format(k) for k in tableau[m - 1, :-1]]
    s += '\t'.join(formattedRow).format(*tableau[m - 1, :-1])
    s += f'|{tableau[m - 1, -1]:6.2f}'
    s += '\n'

    return s

```

```

[5]: def simplexAlgorithmTableau(tableau):
    """
    :param tableau: valid simplex tableau
    :type tableau: numpy.array 2D

    :return: tableau, optimal, unbounded, where tableau is the tableau from the
    → last iteration,
                                     optimal is True if the last tableau is
    → optimal,
                                     unbounded is True if the problem is
    → unbounded.

    :rtype: numpy.array 2D, bool, bool
    """
    while True:
        colPivot, rowPivot, optimal, bounded = simplexTableau(tableau)
        if optimal:
            return tableau, True, False
        if not bounded:
            return tableau, False, True
        tableau = pivoting(tableau, colPivot, rowPivot)

```

```

[6]: def getRow(tableau, index):
    """
    :param tableau: a valid simplex tableau.

```

```

: type tableau: numpy.array 2D

: param index: the index of the variable
: type index: int

: return: row index if the variable is basic, None otherwise
: rtype: int
"""
mtab, ntab = tableau.shape
m = mtab - 1
n = ntab - 1
if index not in range(n):
    raise Exception(f'The index of the variable ({index}) must be between 0_
↳ and {n - 1})')
column = tableau[:, index]
rowIndex = None
for j in range(mtab):
    if np.abs(column[j]) > np.sqrt(np.finfo(float).eps):
        # The entry is non zero
        if rowIndex is None and np.abs(column[j] - 1) <= np.finfo(float).eps:
            # The entry is one, and the index has not been found yet.
            rowIndex = j
        else:
            return None
return rowIndex

```

```

[7]: def simplex(A, b, c):
    """
    : param A: m x n matrix of the constraints
    : type A: numpy.array 2D

    : param b: m vector, left-hand side of the constraints.
    : type b: numpy.array 1D

    : param c: n vector, coefficients of the objective function
    : type c: numpy.array 1D

    : return: tableau, unbounded, infeasible
            where - tableau is the tableau from the last iteration,
                  - unbounded is True if the problem is unbounded,
                  - infeasible is True if the problem is infeasible,
    : rtype: numpy.array 2D, bool, bool
    """

    m, n = A.shape
    if b.shape[0] != m:

```

```

        raise Exception(f'Incompatible sizes: A is {m}x{n}, b is of length {b.
→shape[0]}, and should be {m}')
    if c.shape[0] != n:
        raise Exception(f'Incompatible sizes: A is {m}x{n}, c is of length {c.
→shape[0]}, and should be {n}')

    # All elements of b must be non negative.
    negativeb = np.argwhere(b < 0)
    for i in negativeb:
        A[i, :] = - A[i, :]
        b[i] = - b[i]

    # First tableau for the auxiliary problem
    tableau = np.empty((m + 1, n + m + 1))
    tableau[:m, :n] = A
    tableau[:m, n:n + m] = np.eye(m)
    tableau[:m, n + m:n + m + 1] = b.reshape(m, 1)
    # The last row
    tableau[-1, :n] = np.array([-np.sum(tableau[:m, k]) for k in range(n)]).
→copy()
    tableau[-1, n:n + m] = 0.0
    tableau[-1, -1] = -np.sum(tableau[:m, -1])

    # Solve the auxiliary problem
    phaseOneTableau, optimal, unbounded = simplexAlgorithmTableau(tableau)

    if unbounded:
        return tableau, True, False

    if phaseOneTableau[-1, -1] < -np.sqrt(np.finfo(float).eps):
        # Infeasible problem
        return phaseOneTableau, False, True

    # Remove the auxiliary variables from the basis
    clean = False
    rowsToRemove = []
    basicRows = np.array([getRow(phaseOneTableau, k) for k in range(m + n)])

    while not clean:
        basicIndices = set(np.where(basicRows != None)[0])
        # Check if some auxiliary variables are in the basis
        tobeCleaned = set(basicIndices).intersection(set(range(n, n + m)))
        if tobeCleaned == set():
            clean = True
        else:
            auxiliaryColumn = tobeCleaned.pop()

```

```

        rowpivotIndex = basicRows[auxiliaryColumn]
        rowpivot = phaseOneTableau[rowpivotIndex,:]
        originalNonbasic = list(set(range(n)) - set(basicIndices))
        nonzerosPivots = abs(rowpivot[originalNonbasic]) > np.finfo(float).
→eps

        if nonzerosPivots.any():
            # It is possible to pivot
            colpivot = originalNonbasic[np.argmax(nonzerosPivots)]
            phaseOneTableau = pivoting(phaseOneTableau, colpivot,
→rowpivotIndex)
            basicRows[colpivot] = rowpivotIndex
            basicRows[auxiliaryColumn] = None
        else:
            # It is not possible to pivot. There is a redundant
            # constraint to be removed.
            rowsToRemove.append(rowpivotIndex)
            phaseOneTableau = np.delete(phaseOneTableau, rowsToRemove, 0)
            basicRows = np.array([getRow(phaseOneTableau, k) for k in
→range(m + n)])

            # Delete columns
            startPhaseTwo = np.delete(phaseOneTableau, range(n, n + m), 1)
            m -= len(rowsToRemove)
            basicRows = np.array([getRow(startPhaseTwo, k) for k in range(n)])

            # Calculate last row

            basicIndices = list(np.where(basicRows != None)[0])
            nonbasicIndices = list(np.where(basicRows == None)[0])
            cb = c[basicIndices]
            for k in nonbasicIndices:
                startPhaseTwo[-1, k] = c[k] - np.array([c[j] *
→startPhaseTwo[basicRows[j], k] for j in basicIndices]).sum()
                startPhaseTwo[-1, -1] = - np.array([c[j] * startPhaseTwo[basicRows[j], -1]
→for j in basicIndices]).sum()

            # Phase II

            phaseTwoTableau, optimal, unbounded = simplexAlgorithmTableau(startPhaseTwo)
            return phaseTwoTableau, unbounded, False

```

```

[8]: def getResults(finalTableau):
    mtab, ntab = finalTableau.shape
    m = mtab - 1
    n = ntab - 1
    basicRows = [getRow(finalTableau, k) for k in range(n)]

```

```

        solution = [float(finalTableau[basicRows[k], -1]) if basicRows[k] is not None
→None else 0 for k in range(n)]
        copt = -finalTableau[-1, -1]
        return solution, copt

```

```

[9]: def printResults(res):
    finalTableau, unbounded, infeasible = res
    if unbounded:
        print('Unbounded problem')
        return None, None
    elif infeasible:
        print('Infeasible problem')
        return None, None
    else:
        print(prettyTableau(finalTableau))
        solution, copt = getResults(finalTableau)
        print(f'Optimal solution: {solution}')
        print(f'Optimal value: {copt}')
        return solution, copt

```

2 Branch and bound

This is Algorithm 26.2 in Bierlaire (2015) Optimization: principles and algorithms, EPFL Press.

The following function tests if a real number is actually an integer.

```

[10]: def isInteger(x):
    # We identify a solution as integer if it does not deviate from its
    # rounded version by the square root of the machine epsilon.
    return np.abs(x - np.round(x)) <= np.sqrt(np.finfo(float).eps)

```

The branch and bound algorithm maintains a list of subproblems. The following class implements a subproblem. The key function is the “solve” function. It solves the problem to optimality if it is easy to do so, either because the relaxation generates an integer solution, or because all the subproblems have been solved or discarded.

```

[11]: class subproblem:
    def __init__(self, name, A, b, c, infInequalities, supInequalities):
        self.name = name
        self.A = A
        self.b = b
        self.c = c
        self.m, self.n = A.shape
        self.infInequalities = infInequalities
        self.supInequalities = supInequalities
        self.optimalValue = None
        self.optimalSolution = None

```

```

self.fractionalSolution = None
self.lowerBound = None
self.feasible = None
self.subproblems = None
self.linearOptimization()

def __str__(self):
    c = [f'x[{i}] <= {alpha}' for i, alpha in self.infInequalities.items()]
    c += [f'x[{i}] >= {alpha}' for i, alpha in self.supInequalities.items()]

    if self.optimalValue is not None:
        return f'{self.name} {c} (Opt: {self.optimalValue})'
    elif self.lowerBound is not None:
        return f'{self.name} {c} [LB: {self.lowerBound}]'
    else:
        return f'{self.name} {c}'

def linearOptimization(self):
    p = len(self.infInequalities)
    q = len(self.supInequalities)
    newA = np.concatenate((self.A, np.zeros((self.m, p + q))), 1)
    newA = np.concatenate((newA, np.zeros((p + q, self.n + p + q))))
    newb = np.concatenate((self.b, np.zeros((p + q))))
    newc = np.concatenate((self.c, np.zeros((p + q))))
    i = 0
    for k, alpha in self.infInequalities.items():
        newA[self.m + i, k] = 1.0
        newA[self.m + i, self.n + i] = 1.0
        newb[self.m + i] = alpha
        i += 1
    i = 0
    for k, alpha in self.supInequalities.items():
        newA[self.m + i + p, k] = 1.0
        newA[self.m + i + p, self.n + i + p] = -1.0
        newb[self.m + i + p] = alpha
        i += 1

    tableau, unbounded, infeasible = simplex(newA, newb, newc)
    if unbounded:
        raise Exception('The problem is unbounded.')
    if infeasible:
        print(f'Discard {self} because it is infeasible.')
        self.feasible = False
        self.optimalValue = np.inf
    else:
        print(f'Subproblem {self}')
        self.feasible = True

```



```

        self.fractionalSolution, self.lowerBound = getResults(tableau)
        # Round the solution if "almost" integer
        for i in range(len(self.fractionalSolution)):
            x = self.fractionalSolution[i]
            if np.abs(x - round(x)) <= np.sqrt(np.finfo(float).eps):
                self.fractionalSolution[i] = round(x)

    def solve(self):
        integralSolutions = np.array([isInteger(a) for a in self.
→fractionalSolution])
        if integralSolutions.all():
            # The subproblem is solved to optimality
            self.optimalValue = self.lowerBound
            self.optimalSolution = self.fractionalSolution
            return True

        # If no subproblem has been created, we create them
        if self.subproblems is None:
            p = np.argmax(integralSolutions)
            xfrac = self.fractionalSolution[p]
            P1 = subproblem(f'{self.name}1', A, b, c, {**self.infInequalities, p:
→ np.floor(xfrac)}, self.supInequalities)
            P2 = subproblem(f'{self.name}2', A, b, c, self.infInequalities,
→{**self.supInequalities, p: np.ceil(xfrac)})
            self.subproblems = [P1, P2]
            return False

        bestx = None
        bestValue = np.inf
        # We loop on the subproblems. If they have all been solved, the best
→solution
        # is the optimal solution of the current subproblem.
        # If the optimal value is np.inf, it means that the problem has been
→discarded.
        for p in self.subproblems:
            if p.optimalValue is None:
                # The problem has not been treated. No way to solve the
→subproblem.
                return False
            else:
                if p.optimalValue < bestValue:
                    bestValue = p.optimalValue
                    bestx = p.optimalSolution
        self.optimalValue = bestValue
        self.optimalSolution = bestx
        return True

```

This is the branch and bound algorithm itself. Note that the branching is performed by each subproblem.

```
[12]: def branchAndBound(A, b, c):
    currentBestSol = None
    upperBound = np.inf
    fullProblem = subproblem('P', A, b, c, {}, {})
    setOfProblems = [fullProblem]
    iter = 0
    while setOfProblems:
        iter += 1
        currentProblem = setOfProblems[-1]
        if currentProblem.optimalValue == np.inf:
            setOfProblems.pop()
        else:
            print(f'==== Iteration {iter} =====')
            print(f'Current best solution: {currentBestSol} Value: {upperBound}')
            print(f'Treat problem {currentProblem} - Sol. of relaxation:
→{currentProblem.fractionalSolution}')
            if currentProblem.solve():
                if currentProblem.optimalValue < upperBound:
                    currentBestSol = currentProblem.optimalSolution
                    upperBound = currentProblem.optimalValue
                    # Remove the subproblem from the list
                    print(f'--> Optimal solution of {currentProblem}:
→{currentProblem.optimalSolution}')
                    setOfProblems.pop()
                else:
                    setOfProblems += currentProblem.subproblems
                    msg = [f'\t{p.__str__()} ' for p in setOfProblems if p.optimalValue !
→= np.inf]
                    if len(msg) == 0:
                        print('No more problem to treat.')
                    else:
                        print('List of problems to treat:')
                        print('\n'.join(msg))

                    # Discard problems with a lower bound
                    # worse than the current upperbound
                    for p in setOfProblems:
                        if p.optimalValue != np.inf and p.lowerBound >= upperBound:
                            print(f'Discard {p} because lb={p.lowerBound} >=
→{upperBound}')
                            p.optimalValue = np.inf
    return fullProblem
```

3 Example 1

$$\min -13x_1 - 8x_2$$

subject to

$$\begin{aligned}x_1 + 2x_2 &\leq 10, \\5x_1 + 2x_2 &\leq 20, \\x_1, x_2 &\in \mathbb{N}.\end{aligned}$$

```
[13]: A = np.array([[1, 2, 1, 0], [5, 2, 0, 1]])  
      b = np.array([10, 20])  
      c = np.array([-13, -8, 0, 0])
```

```
[14]: r = branchAndBound(A, b, c)
```

```
Subproblem P []  
===== Iteration 1 =====  
Current best solution: None Value: inf  
Treat problem P [] [LB: -62.5] - Sol. of relaxation: [2.5, 3.75, 0, 0]  
Subproblem P1 ['x[0] <= 2.0']  
Subproblem P2 ['x[0] >= 3.0']  
List of problems to treat:  
    P [] [LB: -62.5]  
    P1 ['x[0] <= 2.0'] [LB: -58.0]  
    P2 ['x[0] >= 3.0'] [LB: -59.0]  
===== Iteration 2 =====  
Current best solution: None Value: inf  
Treat problem P2 ['x[0] >= 3.0'] [LB: -59.0] - Sol. of relaxation: [3, 2.5, 2,  
0, 0]  
Subproblem P21 ['x[1] <= 2.0', 'x[0] >= 3.0']  
Discard P22 ['x[0] >= 3.0', 'x[1] >= 3.0'] because it is infeasible.  
List of problems to treat:  
    P [] [LB: -62.5]  
    P1 ['x[0] <= 2.0'] [LB: -58.0]  
    P2 ['x[0] >= 3.0'] [LB: -59.0]  
    P21 ['x[1] <= 2.0', 'x[0] >= 3.0'] [LB: -57.6]  
===== Iteration 4 =====  
Current best solution: None Value: inf  
Treat problem P21 ['x[1] <= 2.0', 'x[0] >= 3.0'] [LB: -57.6] - Sol. of  
relaxation: [3.2, 2, 2.8, 0, 0, 0.2]  
Subproblem P211 ['x[1] <= 2.0', 'x[0] <= 3.0', 'x[0] >= 3.0']  
Subproblem P212 ['x[1] <= 2.0', 'x[0] >= 4.0']  
List of problems to treat:  
    P [] [LB: -62.5]  
    P1 ['x[0] <= 2.0'] [LB: -58.0]  
    P2 ['x[0] >= 3.0'] [LB: -59.0]  
    P21 ['x[1] <= 2.0', 'x[0] >= 3.0'] [LB: -57.6]  
    P211 ['x[1] <= 2.0', 'x[0] <= 3.0', 'x[0] >= 3.0'] [LB: -55.0]
```

```

P212 ['x[1] <= 2.0', 'x[0] >= 4.0'] [LB: -52.0]
===== Iteration 5 =====
Current best solution: None Value: inf
Treat problem P212 ['x[1] <= 2.0', 'x[0] >= 4.0'] [LB: -52.0] - Sol. of
relaxation: [4, 0, 6, 0, 2, 0]
--> Optimal solution of P212 ['x[1] <= 2.0', 'x[0] >= 4.0'] (Opt: -52.0): [4, 0,
6, 0, 2, 0]
List of problems to treat:
P [] [LB: -62.5]
P1 ['x[0] <= 2.0'] [LB: -58.0]
P2 ['x[0] >= 3.0'] [LB: -59.0]
P21 ['x[1] <= 2.0', 'x[0] >= 3.0'] [LB: -57.6]
P211 ['x[1] <= 2.0', 'x[0] <= 3.0', 'x[0] >= 3.0'] [LB: -55.0]
===== Iteration 6 =====
Current best solution: [4, 0, 6, 0, 2, 0] Value: -52.0
Treat problem P211 ['x[1] <= 2.0', 'x[0] <= 3.0', 'x[0] >= 3.0'] [LB: -55.0] -
Sol. of relaxation: [3, 2, 3, 1, 0, 0, 0]
--> Optimal solution of P211 ['x[1] <= 2.0', 'x[0] <= 3.0', 'x[0] >= 3.0'] (Opt:
-55.0): [3, 2, 3, 1, 0, 0, 0]
List of problems to treat:
P [] [LB: -62.5]
P1 ['x[0] <= 2.0'] [LB: -58.0]
P2 ['x[0] >= 3.0'] [LB: -59.0]
P21 ['x[1] <= 2.0', 'x[0] >= 3.0'] [LB: -57.6]
===== Iteration 7 =====
Current best solution: [3, 2, 3, 1, 0, 0, 0] Value: -55.0
Treat problem P21 ['x[1] <= 2.0', 'x[0] >= 3.0'] [LB: -57.6] - Sol. of
relaxation: [3.2, 2, 2.8, 0, 0, 0.2]
--> Optimal solution of P21 ['x[1] <= 2.0', 'x[0] >= 3.0'] (Opt: -55.0): [3, 2,
3, 1, 0, 0, 0]
List of problems to treat:
P [] [LB: -62.5]
P1 ['x[0] <= 2.0'] [LB: -58.0]
P2 ['x[0] >= 3.0'] [LB: -59.0]
===== Iteration 8 =====
Current best solution: [3, 2, 3, 1, 0, 0, 0] Value: -55.0
Treat problem P2 ['x[0] >= 3.0'] [LB: -59.0] - Sol. of relaxation: [3, 2.5, 2,
0, 0]
--> Optimal solution of P2 ['x[0] >= 3.0'] (Opt: -55.0): [3, 2, 3, 1, 0, 0, 0]
List of problems to treat:
P [] [LB: -62.5]
P1 ['x[0] <= 2.0'] [LB: -58.0]
===== Iteration 9 =====
Current best solution: [3, 2, 3, 1, 0, 0, 0] Value: -55.0
Treat problem P1 ['x[0] <= 2.0'] [LB: -58.0] - Sol. of relaxation: [2, 4, 0, 2,
0]
--> Optimal solution of P1 ['x[0] <= 2.0'] (Opt: -58.0): [2, 4, 0, 2, 0]
List of problems to treat:

```

```

P [] [LB: -62.5]
===== Iteration 10 =====
Current best solution: [2, 4, 0, 2, 0] Value: -58.0
Treat problem P [] [LB: -62.5] - Sol. of relaxation: [2.5, 3.75, 0, 0]
--> Optimal solution of P [] (Opt: -58.0): [2, 4, 0, 2, 0]
No more problem to treat.

Solution:  $x_1 = 2, x_2 = 4$ .

```

4 Example 2

$\min -4x_1 + 6x_2$

subject to

$$\begin{aligned} -4x_1 + 6x_2 &\leq 5, \\ x_1 + x_2 &\leq 5, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

```

[15]: A = np.array([[ -4,  6,  1,  0], [ 1,  1,  0,  1]])
      b = np.array([5, 5])
      c = np.array([1, -2, 0, 0])

```

```

[16]: r = branchAndBound(A, b, c)

```

```

Subproblem P []
===== Iteration 1 =====
Current best solution: None Value: inf
Treat problem P [] [LB: -2.4999999999999996] - Sol. of relaxation:
[2.5000000000000004, 2.5, 0, 0]
Subproblem P1 ['x[0] <= 2.0']
Subproblem P2 ['x[0] >= 3.0']
List of problems to treat:
  P [] [LB: -2.4999999999999996]
  P1 ['x[0] <= 2.0'] [LB: -2.333333333333333]
  P2 ['x[0] >= 3.0'] [LB: -1.0000000000000009]
===== Iteration 2 =====
Current best solution: None Value: inf
Treat problem P2 ['x[0] >= 3.0'] [LB: -1.0000000000000009] - Sol. of
relaxation: [3, 2, 5, 0, 0]
--> Optimal solution of P2 ['x[0] >= 3.0'] (Opt: -1.0000000000000009): [3, 2, 5,
0, 0]
List of problems to treat:
  P [] [LB: -2.4999999999999996]
  P1 ['x[0] <= 2.0'] [LB: -2.333333333333333]
===== Iteration 3 =====
Current best solution: [3, 2, 5, 0, 0] Value: -1.0000000000000009
Treat problem P1 ['x[0] <= 2.0'] [LB: -2.333333333333333] - Sol. of relaxation:
[2, 2.1666666666666665, 0, 0.8333333333333339, 0]

```

```

Subproblem P11 ['x[0] <= 2.0', 'x[1] <= 2.0']
Discard P12 ['x[0] <= 2.0', 'x[1] >= 3.0'] because it is infeasible.
List of problems to treat:
  P [] [LB: -2.4999999999999996]
  P1 ['x[0] <= 2.0'] [LB: -2.333333333333333]
  P11 ['x[0] <= 2.0', 'x[1] <= 2.0'] [LB: -2.25]
===== Iteration 5 =====
Current best solution: [3, 2, 5, 0, 0] Value: -1.0000000000000009
Treat problem P11 ['x[0] <= 2.0', 'x[1] <= 2.0'] [LB: -2.25] - Sol. of
relaxation: [1.7499999999999998, 2, 0, 1.2500000000000009, 0.2500000000000002,
0]
Subproblem P111 ['x[0] <= 1.0', 'x[1] <= 2.0']
Subproblem P112 ['x[0] <= 2.0', 'x[1] <= 2.0', 'x[0] >= 2.0']
List of problems to treat:
  P [] [LB: -2.4999999999999996]
  P1 ['x[0] <= 2.0'] [LB: -2.333333333333333]
  P11 ['x[0] <= 2.0', 'x[1] <= 2.0'] [LB: -2.25]
  P111 ['x[0] <= 1.0', 'x[1] <= 2.0'] [LB: -2.0]
  P112 ['x[0] <= 2.0', 'x[1] <= 2.0', 'x[0] >= 2.0'] [LB: -2.0]
===== Iteration 6 =====
Current best solution: [3, 2, 5, 0, 0] Value: -1.0000000000000009
Treat problem P112 ['x[0] <= 2.0', 'x[1] <= 2.0', 'x[0] >= 2.0'] [LB: -2.0] -
Sol. of relaxation: [2, 2, 1, 1, 0, 0, 0]
--> Optimal solution of P112 ['x[0] <= 2.0', 'x[1] <= 2.0', 'x[0] >= 2.0'] (Opt:
-2.0): [2, 2, 1, 1, 0, 0, 0]
List of problems to treat:
  P [] [LB: -2.4999999999999996]
  P1 ['x[0] <= 2.0'] [LB: -2.333333333333333]
  P11 ['x[0] <= 2.0', 'x[1] <= 2.0'] [LB: -2.25]
  P111 ['x[0] <= 1.0', 'x[1] <= 2.0'] [LB: -2.0]
Discard P111 ['x[0] <= 1.0', 'x[1] <= 2.0'] [LB: -2.0] because lb=-2.0 >= -2.0
===== Iteration 8 =====
Current best solution: [2, 2, 1, 1, 0, 0, 0] Value: -2.0
Treat problem P11 ['x[0] <= 2.0', 'x[1] <= 2.0'] [LB: -2.25] - Sol. of
relaxation: [1.7499999999999998, 2, 0, 1.2500000000000009, 0.2500000000000002,
0]
--> Optimal solution of P11 ['x[0] <= 2.0', 'x[1] <= 2.0'] (Opt: -2.0): [2, 2,
1, 1, 0, 0, 0]
List of problems to treat:
  P [] [LB: -2.4999999999999996]
  P1 ['x[0] <= 2.0'] [LB: -2.333333333333333]
===== Iteration 9 =====
Current best solution: [2, 2, 1, 1, 0, 0, 0] Value: -2.0
Treat problem P1 ['x[0] <= 2.0'] [LB: -2.333333333333333] - Sol. of relaxation:
[2, 2.1666666666666665, 0, 0.8333333333333339, 0]
--> Optimal solution of P1 ['x[0] <= 2.0'] (Opt: -2.0): [2, 2, 1, 1, 0, 0, 0]
List of problems to treat:
  P [] [LB: -2.4999999999999996]

```

```

===== Iteration 10 =====
Current best solution: [2, 2, 1, 1, 0, 0, 0] Value: -2.0
Treat problem P [] [LB: -2.4999999999999996] - Sol. of relaxation:
[2.5000000000000004, 2.5, 0, 0]
--> Optimal solution of P [] (Opt: -2.0): [2, 2, 1, 1, 0, 0, 0]
No more problem to treat.

Solution:  $x_1 = 2, x_2 = 2$ .

```

5 Example 3

$$\min -3x_1 - 13x_2$$

subject to

$$\begin{aligned} 2x_1 + 9x_2 &\leq 29, \\ 11x_1 - 8x_2 &\leq 79, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

```

[17]: A = np.array([[2, 9, 1, 0], [11, -8, 0, 1]])
      b = np.array([29, 79])
      c = np.array([-3, -13, 0, 0])

```

```

[18]: r = branchAndBound(A, b, c)

```

```

Subproblem P []
===== Iteration 1 =====
Current best solution: None Value: inf
Treat problem P [] [LB: -42.8] - Sol. of relaxation: [8.2, 1.4, 0, 0]
Subproblem P1 ['x[0] <= 8.0']
Discard P2 ['x[0] >= 9.0'] because it is infeasible.
List of problems to treat:
  P [] [LB: -42.8]
  P1 ['x[0] <= 8.0'] [LB: -42.77777777777777]
===== Iteration 3 =====
Current best solution: None Value: inf
Treat problem P1 ['x[0] <= 8.0'] [LB: -42.77777777777777] - Sol. of relaxation:
[8, 1.4444444444444442, 0, 2.5555555555555552, 0]
Subproblem P11 ['x[0] <= 8.0', 'x[1] <= 1.0']
Subproblem P12 ['x[0] <= 8.0', 'x[1] >= 2.0']
List of problems to treat:
  P [] [LB: -42.8]
  P1 ['x[0] <= 8.0'] [LB: -42.77777777777777]
  P11 ['x[0] <= 8.0', 'x[1] <= 1.0'] [LB: -36.72727272727273]
  P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] [LB: -42.5]
===== Iteration 4 =====
Current best solution: None Value: inf
Treat problem P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] [LB: -42.5] - Sol. of
relaxation: [5.5, 2, 0, 34.5, 2.5, 0]

```

```

Subproblem P121 ['x[0] <= 5.0', 'x[1] >= 2.0']
Discard P122 ['x[0] <= 8.0', 'x[1] >= 2.0', 'x[0] >= 6.0'] because it is
infeasible.
List of problems to treat:
  P [] [LB: -42.8]
  P1 ['x[0] <= 8.0'] [LB: -42.77777777777777]
  P11 ['x[0] <= 8.0', 'x[1] <= 1.0'] [LB: -36.72727272727273]
  P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] [LB: -42.5]
  P121 ['x[0] <= 5.0', 'x[1] >= 2.0'] [LB: -42.44444444444444]
===== Iteration 6 =====
Current best solution: None Value: inf
Treat problem P121 ['x[0] <= 5.0', 'x[1] >= 2.0'] [LB: -42.44444444444444] -
Sol. of relaxation: [5, 2.111111111111111, 0, 40.888888888888886, 0,
0.1111111111111111]
Subproblem P1211 ['x[0] <= 5.0', 'x[1] <= 2.0', 'x[1] >= 2.0']
Subproblem P1212 ['x[0] <= 5.0', 'x[1] >= 3.0']
List of problems to treat:
  P [] [LB: -42.8]
  P1 ['x[0] <= 8.0'] [LB: -42.77777777777777]
  P11 ['x[0] <= 8.0', 'x[1] <= 1.0'] [LB: -36.72727272727273]
  P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] [LB: -42.5]
  P121 ['x[0] <= 5.0', 'x[1] >= 2.0'] [LB: -42.44444444444444]
  P1211 ['x[0] <= 5.0', 'x[1] <= 2.0', 'x[1] >= 2.0'] [LB: -41.0]
  P1212 ['x[0] <= 5.0', 'x[1] >= 3.0'] [LB: -42.0]
===== Iteration 7 =====
Current best solution: None Value: inf
Treat problem P1212 ['x[0] <= 5.0', 'x[1] >= 3.0'] [LB: -42.0] - Sol. of
relaxation: [1, 3, 0, 92, 4, 0]
--> Optimal solution of P1212 ['x[0] <= 5.0', 'x[1] >= 3.0'] (Opt: -42.0): [1,
3, 0, 92, 4, 0]
List of problems to treat:
  P [] [LB: -42.8]
  P1 ['x[0] <= 8.0'] [LB: -42.77777777777777]
  P11 ['x[0] <= 8.0', 'x[1] <= 1.0'] [LB: -36.72727272727273]
  P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] [LB: -42.5]
  P121 ['x[0] <= 5.0', 'x[1] >= 2.0'] [LB: -42.44444444444444]
  P1211 ['x[0] <= 5.0', 'x[1] <= 2.0', 'x[1] >= 2.0'] [LB: -41.0]
Discard P11 ['x[0] <= 8.0', 'x[1] <= 1.0'] [LB: -36.72727272727273] because
lb=-36.72727272727273 >= -42.0
Discard P1211 ['x[0] <= 5.0', 'x[1] <= 2.0', 'x[1] >= 2.0'] [LB: -41.0] because
lb=-41.0 >= -42.0
===== Iteration 9 =====
Current best solution: [1, 3, 0, 92, 4, 0] Value: -42.0
Treat problem P121 ['x[0] <= 5.0', 'x[1] >= 2.0'] [LB: -42.44444444444444] -
Sol. of relaxation: [5, 2.111111111111111, 0, 40.888888888888886, 0,
0.1111111111111111]
--> Optimal solution of P121 ['x[0] <= 5.0', 'x[1] >= 2.0'] (Opt: -42.0): [1, 3,
0, 92, 4, 0]

```



```

List of problems to treat:
  P [] [LB: -42.8]
  P1 ['x[0] <= 8.0'] [LB: -42.77777777777777]
  P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] [LB: -42.5]
===== Iteration 10 =====
Current best solution: [1, 3, 0, 92, 4, 0] Value: -42.0
Treat problem P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] [LB: -42.5] - Sol. of
relaxation: [5.5, 2, 0, 34.5, 2.5, 0]
--> Optimal solution of P12 ['x[0] <= 8.0', 'x[1] >= 2.0'] (Opt: -42.0): [1, 3,
0, 92, 4, 0]
List of problems to treat:
  P [] [LB: -42.8]
  P1 ['x[0] <= 8.0'] [LB: -42.77777777777777]
===== Iteration 12 =====
Current best solution: [1, 3, 0, 92, 4, 0] Value: -42.0
Treat problem P1 ['x[0] <= 8.0'] [LB: -42.77777777777777] - Sol. of relaxation:
[8, 1.4444444444444442, 0, 2.5555555555555522, 0]
--> Optimal solution of P1 ['x[0] <= 8.0'] (Opt: -42.0): [1, 3, 0, 92, 4, 0]
List of problems to treat:
  P [] [LB: -42.8]
===== Iteration 13 =====
Current best solution: [1, 3, 0, 92, 4, 0] Value: -42.0
Treat problem P [] [LB: -42.8] - Sol. of relaxation: [8.2, 1.4, 0, 0]
--> Optimal solution of P [] (Opt: -42.0): [1, 3, 0, 92, 4, 0]
No more problem to treat.

Solution:  $x_1 = 1, x_2 = 3$ .

```

6 Example 4

$$\min 2x_1 + 3x_2$$

subject to

$$\begin{aligned} 2x_1 + x_2 &\geq 6, \\ x_1 + 3x_2 &\geq 7, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

```

[19]: A = np.array([[2, 1, -1, 0], [1, 3, 0, -1]])
      b = np.array([6, 7])
      c = np.array([2, 3, 0, 0])

```

```

[20]: r = branchAndBound(A, b, c)

```

```

Subproblem P []
===== Iteration 1 =====
Current best solution: None Value: inf
Treat problem P [] [LB: 9.200000000000001] - Sol. of relaxation: [2.2, 1.6, 0,
0]

```

```

Subproblem P1 ['x[0] <= 2.0']
Subproblem P2 ['x[0] >= 3.0']
List of problems to treat:
  P [] [LB: 9.200000000000001]
  P1 ['x[0] <= 2.0'] [LB: 10.0]
  P2 ['x[0] >= 3.0'] [LB: 10.0]
===== Iteration 2 =====
Current best solution: None Value: inf
Treat problem P2 ['x[0] >= 3.0'] [LB: 10.0] - Sol. of relaxation: [3,
1.3333333333333335, 1.3333333333333335, 0, 0]
Subproblem P21 ['x[1] <= 1.0', 'x[0] >= 3.0']
Subproblem P22 ['x[0] >= 3.0', 'x[1] >= 2.0']
List of problems to treat:
  P [] [LB: 9.200000000000001]
  P1 ['x[0] <= 2.0'] [LB: 10.0]
  P2 ['x[0] >= 3.0'] [LB: 10.0]
  P21 ['x[1] <= 1.0', 'x[0] >= 3.0'] [LB: 11.0]
  P22 ['x[0] >= 3.0', 'x[1] >= 2.0'] [LB: 12.0]
===== Iteration 3 =====
Current best solution: None Value: inf
Treat problem P22 ['x[0] >= 3.0', 'x[1] >= 2.0'] [LB: 12.0] - Sol. of
relaxation: [3, 2, 2, 2, 0, 0]
--> Optimal solution of P22 ['x[0] >= 3.0', 'x[1] >= 2.0'] (Opt: 12.0): [3, 2,
2, 2, 0, 0]
List of problems to treat:
  P [] [LB: 9.200000000000001]
  P1 ['x[0] <= 2.0'] [LB: 10.0]
  P2 ['x[0] >= 3.0'] [LB: 10.0]
  P21 ['x[1] <= 1.0', 'x[0] >= 3.0'] [LB: 11.0]
===== Iteration 4 =====
Current best solution: [3, 2, 2, 2, 0, 0] Value: 12.0
Treat problem P21 ['x[1] <= 1.0', 'x[0] >= 3.0'] [LB: 11.0] - Sol. of
relaxation: [4, 1, 3, 0, 0, 1]
--> Optimal solution of P21 ['x[1] <= 1.0', 'x[0] >= 3.0'] (Opt: 11.0): [4, 1,
3, 0, 0, 1]
List of problems to treat:
  P [] [LB: 9.200000000000001]
  P1 ['x[0] <= 2.0'] [LB: 10.0]
  P2 ['x[0] >= 3.0'] [LB: 10.0]
===== Iteration 5 =====
Current best solution: [4, 1, 3, 0, 0, 1] Value: 11.0
Treat problem P2 ['x[0] >= 3.0'] [LB: 10.0] - Sol. of relaxation: [3,
1.3333333333333335, 1.3333333333333335, 0, 0]
--> Optimal solution of P2 ['x[0] >= 3.0'] (Opt: 11.0): [4, 1, 3, 0, 0, 1]
List of problems to treat:
  P [] [LB: 9.200000000000001]
  P1 ['x[0] <= 2.0'] [LB: 10.0]
===== Iteration 6 =====

```

```

Current best solution: [4, 1, 3, 0, 0, 1] Value: 11.0
Treat problem P1 ['x[0] <= 2.0'] [LB: 10.0] - Sol. of relaxation: [2, 2, 0, 1, 0]
--> Optimal solution of P1 ['x[0] <= 2.0'] (Opt: 10.0): [2, 2, 0, 1, 0]
List of problems to treat:
    P [] [LB: 9.200000000000001]
===== Iteration 7 =====
Current best solution: [2, 2, 0, 1, 0] Value: 10.0
Treat problem P [] [LB: 9.200000000000001] - Sol. of relaxation: [2.2, 1.6, 0, 0]
--> Optimal solution of P [] (Opt: 10.0): [2, 2, 0, 1, 0]
No more problem to treat.

Solution:  $x_1 = 2, x_2 = 2$ .

```

7 Example 5

$$\max 8x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 + x_2 &\leq 6, \\ 9x_1 + 5x_2 &\leq 45, \\ x_1, x_2 &\in \mathbb{N}. \end{aligned}$$

```

[21]: A = np.array([[1, 1, 1, 0], [9, 5, 0, 1]])
      b = np.array([6, 45])
      c = np.array([-8, -5, 0, 0])

```

```

[22]: r = branchAndBound(A, b, c)

```

```

Subproblem P []
===== Iteration 1 =====
Current best solution: None Value: inf
Treat problem P [] [LB: -41.25] - Sol. of relaxation: [3.75, 2.25, 0, 0]
Subproblem P1 ['x[0] <= 3.0']
Subproblem P2 ['x[0] >= 4.0']
List of problems to treat:
    P [] [LB: -41.25]
    P1 ['x[0] <= 3.0'] [LB: -39.0]
    P2 ['x[0] >= 4.0'] [LB: -41.0]
===== Iteration 2 =====
Current best solution: None Value: inf
Treat problem P2 ['x[0] >= 4.0'] [LB: -41.0] - Sol. of relaxation: [4, 1.8, 0.19999999999999996, 0, 0]
Subproblem P21 ['x[1] <= 1.0', 'x[0] >= 4.0']
Discard P22 ['x[0] >= 4.0', 'x[1] >= 2.0'] because it is infeasible.
List of problems to treat:
    P [] [LB: -41.25]

```

```

P1 ['x[0] <= 3.0'] [LB: -39.0]
P2 ['x[0] >= 4.0'] [LB: -41.0]
P21 ['x[1] <= 1.0', 'x[0] >= 4.0'] [LB: -40.55555555555556]
===== Iteration 4 =====
Current best solution: None Value: inf
Treat problem P21 ['x[1] <= 1.0', 'x[0] >= 4.0'] [LB: -40.55555555555556] -
Sol. of relaxation: [4.444444444444445, 1, 0.5555555555555556, 0, 0,
0.4444444444444444]
Subproblem P211 ['x[1] <= 1.0', 'x[0] <= 4.0', 'x[0] >= 4.0']
Subproblem P212 ['x[1] <= 1.0', 'x[0] >= 5.0']
List of problems to treat:
P [] [LB: -41.25]
P1 ['x[0] <= 3.0'] [LB: -39.0]
P2 ['x[0] >= 4.0'] [LB: -41.0]
P21 ['x[1] <= 1.0', 'x[0] >= 4.0'] [LB: -40.55555555555556]
P211 ['x[1] <= 1.0', 'x[0] <= 4.0', 'x[0] >= 4.0'] [LB: -37.0]
P212 ['x[1] <= 1.0', 'x[0] >= 5.0'] [LB: -40.0]
===== Iteration 5 =====
Current best solution: None Value: inf
Treat problem P212 ['x[1] <= 1.0', 'x[0] >= 5.0'] [LB: -40.0] - Sol. of
relaxation: [5, 0, 1, 0, 1, 0]
--> Optimal solution of P212 ['x[1] <= 1.0', 'x[0] >= 5.0'] (Opt: -40.0): [5, 0,
1, 0, 1, 0]
List of problems to treat:
P [] [LB: -41.25]
P1 ['x[0] <= 3.0'] [LB: -39.0]
P2 ['x[0] >= 4.0'] [LB: -41.0]
P21 ['x[1] <= 1.0', 'x[0] >= 4.0'] [LB: -40.55555555555556]
P211 ['x[1] <= 1.0', 'x[0] <= 4.0', 'x[0] >= 4.0'] [LB: -37.0]
Discard P1 ['x[0] <= 3.0'] [LB: -39.0] because lb=-39.0 >= -40.0
Discard P211 ['x[1] <= 1.0', 'x[0] <= 4.0', 'x[0] >= 4.0'] [LB: -37.0] because
lb=-37.0 >= -40.0
===== Iteration 7 =====
Current best solution: [5, 0, 1, 0, 1, 0] Value: -40.0
Treat problem P21 ['x[1] <= 1.0', 'x[0] >= 4.0'] [LB: -40.55555555555556] -
Sol. of relaxation: [4.444444444444445, 1, 0.5555555555555556, 0, 0,
0.4444444444444444]
--> Optimal solution of P21 ['x[1] <= 1.0', 'x[0] >= 4.0'] (Opt: -40.0): [5, 0,
1, 0, 1, 0]
List of problems to treat:
P [] [LB: -41.25]
P2 ['x[0] >= 4.0'] [LB: -41.0]
===== Iteration 8 =====
Current best solution: [5, 0, 1, 0, 1, 0] Value: -40.0
Treat problem P2 ['x[0] >= 4.0'] [LB: -41.0] - Sol. of relaxation: [4, 1.8,
0.19999999999999996, 0, 0]
--> Optimal solution of P2 ['x[0] >= 4.0'] (Opt: -40.0): [5, 0, 1, 0, 1, 0]
List of problems to treat:

```

```
P [] [LB: -41.25]
===== Iteration 10 =====
Current best solution: [5, 0, 1, 0, 1, 0] Value: -40.0
Treat problem P [] [LB: -41.25] - Sol. of relaxation: [3.75, 2.25, 0, 0]
--> Optimal solution of P [] (Opt: -40.0): [5, 0, 1, 0, 1, 0]
No more problem to treat.

Solution:  $x_1 = 5, x_2 = 0$ .
```